

TD3 : Interrogations en SQL — Corrigé b37eca9

Rosine Cicchetti - Lotfi Lakhal - Mickaël Martin Nevot



Cette œuvre de Rosine Cicchetti est mise à disposition sous licence Creative Commons
Attribution - Utilisation non commerciale - Partage dans les mêmes conditions.

Document en ligne : www.mickael-martin-nevot.com

1 Rappel : Prise en main d'un éditeur Oracle

Utilisez une des deux solutions ci-dessous.

1.1 Oracle Live SQL

Vous pouvez utiliser directement, sans installation ou configuration, l'interpréteur en ligne :
<https://livesql.oracle.com>.

1.2 Oracle Cloud Free Tier

Mettez en place une solution d'hébergement en ligne d'un SGBD Oracle. Vous pouvez pour cela consulter le document [Vade-Mecum mise en place d'un hébergement Oracle Cloud Free Tier](#).

Ajouter ensuite une base de données nommée `zenetude-bd`.

2 Rappel : Généralités

Voici la convention de nommage proposé dans cet enseignement :

- **mots clefs** : en lettre capitales (*upper case*) ;
- **relation** : première lettre de chaque mot en capitale (*pascal case*) ;
- **attributs** : premier mot en minuscule et première lettre de chaque mot suivant en capitale (*camel case*) ;
- **domaine** : en lettre capitales (*upper case*) au format D_XXX¹.

Pour rappel, les valeurs saisies dans une base de données, comme les chaînes de caractères, sont sensibles à la casse et, par convention, sont saisies **en lettres capitales** (*upper case*), sans diacritique, dans le cadre de cet enseignement.

1. Les domaines identiques sont surlignés avec la même couleur.

3 Base de données exemple

La base de données exemple, Gestion pédagogique, utilisée dans les documents de travaux dirigés de cet enseignement, permet de gérer une formation en informatique, avec ses professeurs, ses étudiants, ses modules ainsi que leurs enseignements et notations.

3.1 Dictionnaire de données

La base de données Gestion pédagogique a été élaborée à partir du dictionnaire des données suivant² :

Table 1 – Dictionnaire des données de la base de données exemple

Attribut	Description	Domaine	Remarques
numEt	Numéro d'un étudiant	D_NUMET : NUMBER(6,0)	Valeurs uniques
nomEt	Nom d'un étudiant	D_NOM : VARCHAR2(30)	
prenomEt	Prénom d'un étudiant	D_PRENOM : VARCHAR2(20)	
cpEt	Code postal d'un étudiant	D_CP : VARCHAR2(5)	
villeEt	Ville d'un étudiant	D_VILLE : VARCHAR2(20)	
anneeEt	Année d'étude d'un étudiant	D_ANNEE : NUMBER(2,0)	[1..2]
groupeEt	Numéro de groupe d'un étudiant	D_GROUPE : NUMBER(1,0)	[1..5]
numProf	Numéro d'un professeur	D_NUMPROF : NUMBER(3,0)	Valeurs uniques
nomProf	Nom d'un professeur	D_NOM : VARCHAR2(20)	
prenomProf	Prénom d'un professeur	D_PRENOM : VARCHAR2(20)	
adresseProf	Adresse d'un professeur	D_ADRESSE : VARCHAR2(40)	
cpProf	Code postal d'un professeur	D_CP : VARCHAR2(5)	
villeProf	Ville d'un professeur	D_VILLE : VARCHAR2(20)	
specProf	Code d'une matière dont un professeur est spécialiste	D_CODE : VARCHAR2(7)	
code	Code d'un module	D_CODE : VARCHAR2(7)	Valeurs uniques
libelle	Libellé d'un module	D_LIBELLE : VARCHAR2(30)	
heureCMPrev	Nombre d'heures de CM prévues pour un module	D_NBHEURE : NUMBER(3,0)	

2. Les types syntaxiques, utilisés pour la description des domaines, sont disponibles dans Oracle.

Attribut	Description	Domaine	Remarques
heureCMReal	Nombre d'heures de CM réalisées pour un module	D_NBHEURE : NUMBER(3,0)	
heureTPPrev	Nombre d'heures de TP prévues pour un module	D_NBHEURE : NUMBER(3,0)	
heureTPReal	Nombre d'heures de TP réalisées pour un module	D_NBHEURE : NUMBER(3,0)	
discipline	Discipline enseignée	D_DISCIPLINE : VARCHAR2(15)	{INFORMATIQUE, MATHS, GESTION}
coefCC	Coefficient du contrôle continu pour un module	D_COEF : NUMBER(3,0)	[0..100]
coefTest	Coefficient du test pour un module	D_COEF : NUMBER(3,0)	[0..100]
resp	Numéro d'un professeur responsable d'une matière	D_NUMPROF : NUMBER(3,0)	
codePere	Code du module père d'un module	D_CODE : VARCHAR2(7)	
moyCC	Moyenne de contrôle continu d'un étudiant dans un module	D_NOTE : NUMBER(2,0)	[0..20]
moyTest	Moyenne de test d'un étudiant dans un module	D_NOTE : NUMBER(2,0)	[0..20]

Remarques

Les modules forment une hiérarchie entre eux. Quel que soit leur niveau dans la hiérarchie, tous les modules possèdent les mêmes attributs. La racine de l'arbre ainsi formé par cette hiérarchie est le programme pédagogique national de formation en informatique. Il se décompose en véritables modules, eux-mêmes se décomposant en sous-modules et ainsi de suite jusqu'aux feuilles qui correspondent aux matières. Les matières sont les seuls modules pouvant être enseignés et susceptibles de recevoir des notes. Certains modules sont associés à une discipline.

La hiérarchie des modules de l'extension de la base de données exemple est représentée sous forme d'arborescence des codes en figure 1.

Les coefficients de contrôle continue et de test d'un module sont des pourcentages. Ils peuvent ne pas être renseignés. Si un des deux coefficients n'est pas renseigné, l'autre ne doit pas l'être non plus. Leur somme pour un même module est donc soit de 0, soit de 100.

Nous considérons qu'il n'y a pas deux personnes homonymes ayant le même prénom et le même nom.

3.2 MCD

Le MCD (voir figure 2) modélise trois entités : **Module** (les unités d'enseignement organisées en hiérarchie), **Étudiant** et **Prof**. L'association **Notation** relie un étudiant à un module et porte ses moyennes de contrôle continu et de test. L'association ternaire **Enseignement** lie un



Figure 1 – Hiérarchie des modules

professeur, un module et un étudiant. Un professeur peut être **Spécialiste** d'un module (sa discipline de référence) et **Responsable** d'un module. Enfin, l'association réflexive **A pour père** matérialise la hiérarchie entre modules (voir figure 1).



Figure 2 – Modèle conceptuel des données (MCD)

3.3 Schéma relationnel

Les clefs primaires sont soulignées et les clefs étrangères en *italique suivies d'un #*.

Le schéma relationnel de la base de données exemple est présenté ci-après :

Etudiant (numEt, nomEt, prenomEt, cpEt, villeEt, anneeEt, groupeEt)

Professeur (numProf, nomProf, prenomProf, adresseProf, cpProf, villeProf, *specProf#*)

Module (code, libelle, heureCMPprev, heureCMReal, heureTDprev, heureTDReal, discipline, co-

efCC, coefTest, resp#, codePere#)

Note (numEt#, code#, moyCC, moyTest)

Enseigne (numEt#, code#, numProf#)

Remarques

La clef étrangère **resp** dans **Module** représente le numéro d'un professeur responsable d'un module^a et fait référence à la clef primaire **numProf**.

La clef étrangère **specProf** dans **Professeur** représente le code d'une matière dont un professeur est spécialiste et fait référence à la clef primaire **code**.

La clef étrangère **codePere** dans la relation **Module** fait référence à la clef primaire **code**.

^a. Idéalement, il ne devrait être possible d'y avoir qu'un responsable d'une matière, pas de n'importe quel module. Par souci de simplification, cette contrainte n'a pas été prise en compte dans l'intension de la base de données exemple.

Les autres clefs étrangères font référence aux clefs primaires de même nom.

4 Requêtes avec SQL

Formulez, en SQL, sur la base de données exemple, les requêtes d'interrogation suivantes.

4.1 Expression des jointures

Quand cela possible, formulez les requêtes suivantes de trois manières différentes.

Q1 Donnez, pour l'étudiant Stéphane Rocchi, les moyennes de test obtenues par ordre décroissant avec le code du module associé. *2 attributs, 9 tuples*

Version algébrique.

```
SELECT code, moyTest
FROM Etudiant Et
  INNER JOIN Note N ON Et.numEt = N.numEt
WHERE
  nomEt = 'ROCCHI'
  AND prenomEt = 'STEPHANE'
ORDER BY moyTest DESC;
```

Version imbriquée.

```
SELECT code, moyTest
FROM Note
WHERE
  numEt IN (
    SELECT numEt
    FROM Etudiant
    WHERE
      nomEt = 'ROCCHI'
      AND prenomEt = 'STEPHANE'
  )
ORDER BY moyTest DESC;
```

Version prédicative.

```
SELECT code, moyTest
FROM Etudiant Et
     INNER JOIN Note N ON Et.numEt = N.numEt
WHERE
    nomEt = 'ROCCHI'
    AND prenomEt = 'STEPHANE'
ORDER BY moyTest DESC;
```

Q2 Donnez les codes et libellés des modules enseignés par Didier Boitard. *2 attributs, 2 tuples*

Version algébrique.

```
SELECT DISTINCT M.code, libelle
FROM Module M
     INNER JOIN Enseigne E ON M.code = E.code
     INNER JOIN Professeur P ON E.numProf = P.numProf
WHERE
    nomProf = 'BOITARD'
    AND prenomProf = 'DIDIER';
```

Version imbriquée.

```
SELECT code, libelle
FROM Module
WHERE code IN (
    SELECT code
    FROM Enseigne
    WHERE numProf IN (
        SELECT numProf
        FROM Professeur
        WHERE
            nomProf = 'BOITARD'
            AND prenomProf = 'DIDIER'
    )
);
```

Version prédicative.

```
SELECT DISTINCT M.code, libelle
FROM Module M, Professeur P, Enseigne E
WHERE
    M.code = E.code
    AND E.numProf = P.numProf
    AND nomProf = 'BOITARD'
    AND prenomProf = 'DIDIER';
```

Remarques

Dans la version algébrique et prédicative, il y a autant de tuples résultats que d'étudiants ayant un enseignement de Didier Boitard. Il faut donc utiliser DISTINCT pour éliminer les modules dupliqués. Dans la version imbriquée, le premier bloc n'opère que sur la relation module dont code est la clef primaire. Il ne peut donc pas y avoir de duplicats et DISTINCT n'a donc pas besoin d'être utilisé.

Q3 Quels sont les groupes de seconde année pour lesquels Marc Laporte a effectué un enseignement ? *1 attribut, 2 tuples*

Version algébrique.

```
SELECT DISTINCT groupeEt
FROM Etudiant Et
    INNER JOIN Enseigne E ON Et.numEt = E.numEt
    INNER JOIN Professeur P ON E.numProf = P.numProf
WHERE
    anneeEt = 2
    AND nomProf = 'LAPORTE'
    AND prenomProf = 'MARC';
```

Version imbriquée.

```
SELECT DISTINCT groupeEt
FROM Etudiant
WHERE
    anneeEt = 2
    AND numEt IN
    (
        SELECT numEt
        FROM Enseigne
        WHERE
            numProf IN
            (
                SELECT numProf
                FROM Professeur
                WHERE
                    nomProf = 'LAPORTE'
                    AND prenomProf = 'MARC'
            )
    );
```

Version prédicative.

```
SELECT DISTINCT groupeEt
FROM Etudiant Et, Enseigne E, Professeur P
WHERE
    Et.numEt = E.numEt
    AND E.numProf = P.numProf
    AND anneeEt = 2
    AND nomProf = 'LAPORTE'
    AND prenomProf = 'MARC';
```

Remarques

Quelle que soit la version de la requête, il y a autant de valeurs résultats que d'étudiants ayant un enseignement de Marc Laporte. Il faut donc toujours utiliser DISTINCT pour éliminer les numéros de groupeEt dupliqués.

Q4 Donnez les numéros et, par ordre alphabétique, les noms des étudiants ayant suivi un enseignement effectué par un professeur, par ailleurs responsable d'un module quelconque. *2 attributs, 18 tuples*

Version algébrique.

```
SELECT DISTINCT Et.numEt, nomEt
FROM Etudiant Et
     INNER JOIN Enseigne E ON Et.numEt = E.numEt
     INNER JOIN Module M ON E.numProf = resp
ORDER BY numEt ASC;
```

Version imbriquée.

```
SELECT numEt, nomEt
FROM Etudiant
WHERE
    numEt IN
    (
        SELECT numEt
        FROM Enseigne
        WHERE numProf IN (
            SELECT resp
            FROM Module
        )
    )
ORDER BY numEt ASC;
```

Version prédictive.

```
SELECT DISTINCT Et.numEt, nomEt
FROM Etudiant Et, Enseigne E, Module M
WHERE
    Et.numEt = E.numEt
    AND E.numProf = resp
ORDER BY numEt ASC;
```

Remarques

Un étudiant pouvant avoir plusieurs fois le même professeur pour des enseignements différents, le mot-clef DISTINCT est nécessaire pour éliminer les duplicats, cependant pour conserver les homonymes dans le résultat, il convient d'effectuer la projection de l'attribut clef primaire numEt.

Q5 Donnez les numéros et, par ordre alphabétique, les noms des étudiants ayant suivi un enseignement effectué par le professeur responsable de ce module. *2 attributs, 12 tuples*

Version algébrique.

```

SELECT DISTINCT Et.numEt, nomEt
FROM Etudiant Et
    INNER JOIN Enseigne E ON Et.numEt = E.numEt
    INNER JOIN Module M
        ON
            E.code = M.code
            AND numProf = resp
ORDER BY nomEt ASC;

```

Version imbriquée.

```

SELECT numEt, nomEt
FROM Etudiant
WHERE
    numEt IN (
        SELECT numEt
        FROM Enseigne
        WHERE (numProf, code) IN (
            SELECT resp, code FROM Module
        )
    )
ORDER BY nomEt ASC;

```

Version prédicative.

```

SELECT DISTINCT Et.numEt, nomEt
FROM Etudiant Et, Enseigne E, Module M
WHERE
    Et.numEt = E.numEt
    AND E.code = M.code
    AND numProf = resp
ORDER BY nomEt ASC;

```

4.2 Formulation de calculs verticaux et horizontaux

Formulez les requêtes suivantes, en veillant particulièrement à l'évaluation des valeurs nulles pour les calculs horizontaux.

Q6 Combien y-a-t-il de professeurs? *1 attribut, 1 tuple (17)*

```

SELECT COUNT(*)
FROM Professeur;

```

Q7 Quelle est la moyenne générale des notes de contrôle continu pour le module de code PRL ?
1 attribut, 1 tuple (9.7)

```

SELECT AVG(moyCC)
FROM Note
WHERE code = 'PRL';

```

Q8 Combien de professeurs ont donné un enseignement à l'étudiant Philippe Lyon ? *1 attribut, 1 tuple (7)*

```
SELECT COUNT(DISTINCT numProf)
FROM Enseigne E
      INNER JOIN Etudiant Et ON E.numEt = Et.numEt
WHERE
      nomEt = 'LYON'
      AND prenomEt = 'PHILIPPE';
```

Remarques

Sans l'utilisation de DISTINCT, la requête ne retourne pas le résultat voulu mais le nombre d'enseignements reçus par l'étudiant Philippe Lyon. S'il se trouve que cet étudiant n'a pas plusieurs fois le même professeur, la requête sans DISTINCT donnera un résultat juste mais elle est néanmoins logiquement fausse.

Q9 Donnez, pour le module Prolog, la note moyenne obtenue par les étudiants en tenant compte des coefficients de contrôle continu et de test. *1 attribut, 1 tuple (10.66)*

```
SELECT AVG((COALESCE(moyCC, 0) * coefCC + COALESCE(moyTest, 0) * coefTest) / (coefCC + coefTest))
FROM Note N
      INNER JOIN Module M ON N.code = M.code
WHERE libelle = 'PROLOG';
```

Remarques

L'alias moyenne spécifié après l'expression de calcul horizontal est introduit pour, notamment, éviter l'affichage de l'expression.

Q10 Quel est le coefficient de test le plus faible ? *1 attribut, 1 tuple (0)*

```
SELECT MIN(coefTest)
FROM Module;
```

Q11 Quels sont les libellés des modules dont le coefficient de test est le plus faible ? Proposer deux formulations différentes de cette requête. *1 attribut, 1 tuple (ETUDE DE CAS 1)*

V1.

```
SELECT DISTINCT libelle
FROM Module
WHERE coefTest = (
      SELECT MIN(coefTest)
      FROM Module
);
```

V2.

```
SELECT DISTINCT libelle
FROM Module
```

```
WHERE coefTest <= ALL(
    SELECT coefTest
    FROM Module
    WHERE coefTest IS NOT NULL
);
```

Remarques

Dans la seconde requête, la condition `coefTest IS NOT NULL` doit être spécifiée. En effet si l'attribut concerné comprend des valeurs nulles, le résultat de la sous-requête est évalué à « inconnu » et la requête principale ne rend aucun tuple.

Q12 En supposant que tous les modules sont de même importance, donnez la moyenne générale de l'étudiante Sandrine Levy. *1 attribut, 1 tuple (11.13)*

```
SELECT AVG(((COALESCE(moyCC, 0) * coefCC) + (COALESCE(moyTest, 0) * coefTest)) / (coefCC + coefTest))
FROM Module M
    INNER JOIN Note N ON M.code = N.code
    INNER JOIN Etudiant Et ON N.numEt = Et.numEt
    AND nomEt = 'LEVY'
    AND prenomEt = 'SANDRINE';
```

Q13 Quels sont les codes des modules pour lesquels la meilleure note de test a été obtenue ? *1 attribut, 2 tuples*

```
SELECT DISTINCT code
FROM Note
WHERE moyTest = (
    SELECT MAX(moyTest)
    FROM Note
);
```

Q14 Quels sont les numéros et noms des étudiants qui ont obtenu, tous modules confondus, la meilleure note de test ? Proposer deux formulations différentes de cette requête. *2 attributs, 2 tuples*

V1.

```
SELECT DISTINCT Et.numEt, nomEt
FROM Etudiant Et
    INNER JOIN Note N ON Et.numEt = N.numEt
WHERE moyTest = (
    SELECT MAX(moyTest)
    FROM Note
);
```

Remarques

Il faut procéder à l'élimination des duplicats (par `DISTINCT`) car le même étudiant peut avoir obtenu la note maximale dans plusieurs modules différents.

V2.

```

SELECT DISTINCT Et.numEt, nomEt
FROM Etudiant Et
     INNER JOIN Note N ON Et.numEt = N.numEt
WHERE moyTest >= ALL(
    SELECT moyTest
    FROM Note
    WHERE moyTest IS NOT NULL
);

```

Remarques

La condition de sélection moyTest IS NOT NULL, dans la requête imbriquée, est nécessaire car il suffit d'une note de test inconnue pour que la requête ne rende aucun résultat.

V3.

```

SELECT numEt, nomEt
FROM Etudiant
WHERE numEt IN (
    SELECT numEt
    FROM Note
    WHERE moyTest IN (
        SELECT MAX(moyTest)
        FROM Note
    )
);

```

4.3 Utilisation des opérateurs ensemblistes**Q15** Donnez les villes de résidence des étudiants et des professeurs. *1 attribut, 6 tuples*

```

SELECT villeEt
FROM Etudiant
WHERE villeEt IS NOT NULL
UNION
SELECT villeProf
FROM Professeur
WHERE villeProf IS NOT NULL;

```

Q16 Quels sont les numéros des professeurs responsables d'un module qu'ils enseignent ainsi que le code du module correspondant ? *2 attributs, 4 tuples*

```

SELECT resp, code
FROM Module
INTERSECT
SELECT numProf, code
FROM Enseigne;

```

Q17 Donnez les libellés des modules ne correspondant à la spécialité d'aucun professeur. *1 attribut, 23 tuples*

```
SELECT DISTINCT libelle
FROM Module
WHERE code IN (
    SELECT code
    FROM Module
    EXCEPT
    SELECT specProf
    FROM Professeur
);
```

4.4 Équivalent des opérateurs ensemblistes

Proposez une nouvelle formulation des requêtes de la section précédente sans faire appel aux opérateurs ensemblistes.

Q18 Quels sont les numéros des professeurs responsables d'un module qu'ils enseignent ainsi que le code du module correspondant ? *Q16, 2 attributs, 4 tuples*

V1.

```
SELECT DISTINCT resp, M.code
FROM Module M
    INNER JOIN Enseigne E
        ON
            M.resp = E.numProf
        AND M.code = E.code;
```

Remarques

Le mot clef DISTINCT est obligatoire car un même responsable peut enseigner le même module à des étudiants différents.

V2.

```
SELECT resp, code
FROM Module
WHERE (
    resp, code) IN (
    SELECT numProf, code
    FROM Enseigne
);
```

Remarques

Il est inutile de contrôler que la sous-requête puisse retourner des valeurs nulles. En effet, ce cas est impossible : numProf et code font partie de la clef primaire de Enseigne.

V3.

```
SELECT resp, code
FROM Module M
WHERE EXISTS (
```

```

SELECT *
FROM Enseigne E
WHERE
    E.numProf = M.resp -- noqa: RF03
    AND E.code = M.code
);

```

Remarques

Avec EXISTS, la sous-requête est corrélée : elle fait référence à la table Module du bloc externe. Pour chaque ligne de Module, la sous-requête vérifie s'il existe au moins un enseignement correspondant. Le mot clef DISTINCT n'est pas nécessaire car la projection porte sur la clef primaire de Module (code) et un attribut fonctionnellement dépendant (resp).

Q19 Donnez les libellés des modules ne correspondant à la spécialité d'aucun professeur. *Q17, 1 attribut, 23 tuples*

V1.

```

SELECT DISTINCT libelle
FROM Module
WHERE code NOT IN (
    SELECT specProf
    FROM Professeur
    WHERE specProf IS NOT NULL
);

```

Remarques

Il faut spécifier la condition NOT NULL, car le prédicat « NOT IN » est évalué à « inconnu » quand l'ensemble retourné par la sous-requête comporte une valeur nulle.

V2.

```

SELECT DISTINCT libelle
FROM Module M
WHERE NOT EXISTS (
    SELECT *
    FROM Professeur
    WHERE specProf = M.code -- noqa: RF03
);

```

V3.

```

SELECT DISTINCT libelle
FROM Module
    LEFT OUTER JOIN Professeur ON code = specProf
WHERE specProf IS NULL;

```

4.5 Test d'absence de données

Formulez les requêtes suivantes de trois manières différentes, sans utiliser d'opérateur ensembliste.

Q20 Donnez les numéros, noms et prénoms des étudiants n'ayant aucune note. *3 attributs, 27 tuples*

V1.

```
SELECT numEt, nomEt, prenomEt
FROM Etudiant
WHERE numEt NOT IN (
    SELECT numEt
    FROM Note
);
```

V2.

```
SELECT numEt, nomEt, prenomEt
FROM Etudiant
WHERE numEt <> ALL(
    SELECT numEt
    FROM Note
);
```

V3.

```
SELECT numEt, nomEt, prenomEt
FROM Etudiant E
WHERE NOT EXISTS (
    SELECT *
    FROM Note N
    WHERE N.numEt = E.numEt -- noqa: RF03
);
```

V4.

```
SELECT Et.numEt, nomEt, prenomEt
FROM Etudiant Et
    LEFT OUTER JOIN Note N ON Et.numEt = N.numEt
WHERE N.numEt IS NULL;
```

Q21 Quels sont les noms et prénoms des étudiants n'ayant eu aucun enseignement de Marc Laporte? *2 attributs, 41 tuples*

V1.

```
SELECT nomEt, prenomEt
FROM Etudiant
WHERE numEt NOT IN (
    SELECT numEt
    FROM Enseigne E
        INNER JOIN Professeur P ON E.numProf = P.numProf
    WHERE
        nomProf = 'LAPORTE'
        AND prenomProf = 'MARC'
);
```

V2.

```

SELECT nomEt, prenomEt
FROM Etudiant
WHERE numEt <> ALL(
    SELECT numEt
    FROM Enseigne E
    INNER JOIN Professeur P ON E.numProf = P.numProf
    WHERE
        nomProf = 'LAPORTE'
        AND prenomProf = 'MARC'
);

```

V3.

```

SELECT nomEt, prenomEt
FROM Etudiant Et
WHERE NOT EXISTS (
    SELECT *
    FROM Enseigne E
    INNER JOIN Professeur P ON E.numProf = P.numProf
    WHERE
        E.numEt = Et.numEt
        AND nomProf = 'LAPORTE'
        AND prenomProf = 'MARC'
);

```

V4.

```

SELECT nomEt, prenomEt
FROM Etudiant Et
    LEFT OUTER JOIN (
        SELECT numEt
        FROM Enseigne E
        INNER JOIN Professeur P ON E.numProf = P.numProf
        WHERE
            nomProf = 'LAPORTE'
            AND prenomProf = 'MARC'
    ) T
    ON Et.numEt = T.numEt
WHERE T.numEt IS NULL;

```

Q22 Quels sont les noms et prénoms des professeurs n'étant pas responsables de modules? 2
attributs, 8 tuples

V1.

```

SELECT nomProf, prenomProf
FROM Professeur
WHERE numProf NOT IN (
    SELECT resp

```



```
FROM Module
WHERE resp IS NOT NULL
);
```

V2.

```
SELECT nomProf, prenomProf
FROM Professeur
WHERE numProf <> ALL(
    SELECT resp
    FROM Module
    WHERE resp IS NOT NULL
);
```

Remarques

Dans les deux versions précédente de la requête, la condition de sélection « resp IS NOT NULL » est nécessaire. Sans elle, l'attribut resp ayant des valeurs nulles, le résultat de la sous-requête est évalué à « inconnu » et la requête ne rend aucun tuple.

V3.

```
SELECT nomProf, prenomProf
FROM Professeur P
WHERE NOT EXISTS (
    SELECT *
    FROM Module M
    WHERE M.resp = P.numProf -- noqa: RF03
);
```

V4.

```
SELECT nomProf, prenomProf
FROM Professeur
    LEFT OUTER JOIN Module ON numProf = resp
WHERE resp IS NULL;
```

4.6 Expression des partitionnements

Q23 Donnez, par groupe de seconde année, le nombre d'étudiants. *2 attributs, 5 tuples*

```
SELECT groupeEt, COUNT(*) AS nbEtudiants
FROM Etudiant
WHERE anneeEt = 2
GROUP BY groupeEt;
```

Q24 Donnez, pour chaque numéro d'étudiant, la meilleure note de test. *2 attributs, 29 tuples*

```
SELECT numEt, MAX(moyTest) AS maxMoyTest
FROM Note
GROUP BY numEt;
```

Q25 Donnez, pour chaque numéro et nom des étudiants de seconde année ainsi que pour chaque code de module, le nombre de professeurs. *4 attributs, 71 tuples*

```
SELECT Et.numEt, nomEt, code, COUNT(numProf) AS nbProfs
FROM Etudiant Et
     INNER JOIN Enseigne E ON Et.numEt = E.numEt
WHERE anneeEt = 2
GROUP BY Et.numEt, nomEt, code;
```

Remarques

La projection de l'attribut clef primaire numEt assure que des fragments différents sont créés par le partitionnement même en cas d'homonymie des étudiants. il est aussi inutile de préciser DISTINCT avant l'argument de la fonction COUNT. En effet, par définition de la relation Enseigne, pour une matière et un étudiant donnés, un numéro de professeur n'apparaît qu'une seule fois.

Q26 Donnez, pour chaque ville de plus de cinq professeurs, le nombre de professeurs y résidant. *2 attributs, 2 tuples*

```
SELECT villeProf, COUNT(*) AS nbProfs
FROM Professeur
GROUP BY villeProf
HAVING COUNT(*) > 5;
```